Case Study Report

on

Automating Docker Image Build and Push to Docker Hub using Jenkins

Bachelor of Technology
in
Computer Science Engineering (Artificial Intelligence & Machine Learning)

By

Student Name	**Roll No**
Gunna Pavan	A23126552265
Nalli Bobby Raj	A23126552267
Nazeer Ahmad	A23126552269
Rapaka Dinesh	A23126552271
Mondi Naga Durga Ramesh	A23126552272

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (AI & ML)

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

Sangivalasa, Bheemunipatnam Mandal,

Visakhapatnam Dist, AP, India

2026

CERTIFICATE

The case study report entitled **"Automating Docker Image Build and Push to Docker Hub using Jenkins"** submitted by Gunna Pavan, Nalli Bobby Raj, Nazeer Ahmad, Rapaka Dinesh, Mondi Naga Durga Ramesh III/IV B.Tech, II-Semester bearing Regd. No. A23126552265, A23126552267, A23126552269, A23126552271, A23126552272 Dept. of CSE(AI&ML) is found to be satisfactory.

Mr. Santosh Kumar P
Associate Professor
Dept. of CSE(AI&ML)

ACKNOWLEDGEMENT

This case study reflects our academic learning and professional development journey. The case study provided an opportunity to apply classroom knowledge in a real-world cloud and containerization environment.

We express sincere gratitude to Dr. K. Selvani Deepthi, Head of the Department of Computer Science and Engineering (AI & ML) at ANITS, and our guide Mr. Santosh Kumar P, Associate Professor, for invaluable guidance and encouragement throughout the case study. We also acknowledge the faculty members for their unwavering support.

Regd No. A23126552265
A23126552267
A23126552269
A23126552271
A23126552272

CONTENTS

1. **Abstract** 5
2. **Introduction to DevOps** 5
3. **Objectives of the Project** 6
4. **Tools and Technologies Used** 6
5. **System Architecture** 7
6. **Prerequisites** 8
7. **Step-by-Step Implementation** 8
8. **Project Files Explanation** 10
9. **Output Explanation** 12
10. **Advantages of the Project** 15
11. **Limitations** 15
12. **Conclusion** 16
13. **Future Enhancements** 16
14. **References** 17

1. Abstract

This project demonstrates the design and implementation of a Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins, Docker, and Docker Hub on an AWS EC2 instance. The objective is to automate the process of building a Docker image from application source code and deploying a running container — entirely without manual intervention.

The application is a simple HTML web page served using Nginx. Jenkins builds a Docker image, logs in to Docker Hub securely, pushes the image, and then automatically stops any old container and runs the updated one. This project illustrates core DevOps principles of automation and serves as a practical foundation for understanding CI/CD pipelines in real-world software development.

2. Introduction to DevOps

DevOps is a cultural and technical movement that bridges the gap between software development (Dev) and IT operations (Ops). It promotes collaboration, automation, and continuous delivery to improve the speed and quality of software releases. Modern development teams rely heavily on DevOps practices to reduce errors, speed up deployments, and ensure reliable software delivery.

2.1 Key Principles of DevOps

- Continuous Integration (CI): Frequently merging code changes and automatically building and testing them to detect issues early.
- Continuous Delivery (CD): Automatically delivering and deploying tested code to production or staging environments.
- Infrastructure as Code: Managing servers and environments using version-controlled scripts.
- Monitoring and Feedback: Continuously observing systems to improve reliability and performance.

2.2 Importance of CI/CD Pipelines

A CI/CD pipeline automates every step from writing code to running it in production. This reduces human errors, speeds up delivery, and ensures every build is tested and traceable. Jenkins is one of the most widely used open-source CI/CD tools enabling the creation of such automated pipelines.

Docker allows developers to package applications with all their dependencies into portable containers. When combined with Jenkins, Docker enables consistent and reproducible builds across any environment — from a developer's machine to a cloud server.

3. Objectives of the Project

The main objectives of this project are:

1. To set up Jenkins on an AWS EC2 instance for automating CI/CD tasks.
2. To install and configure Docker on the same EC2 server.
3. To create a Jenkins pipeline that automatically builds a Docker image from code.
4. To securely push the built Docker image to Docker Hub using Jenkins credentials.
5. To automatically deploy the updated container on the same server after every push.
6. To use a Jenkinsfile for pipeline-as-code configuration.
7. To serve a simple HTML web page using an Nginx-based Docker container.

4. Tools and Technologies Used

Tool / Technology	Purpose
AWS EC2 (Ubuntu Linux)	Cloud server used to host Jenkins and Docker. Ubuntu 22.04 LTS was used.
Jenkins	Open-source automation server used to create and run the CI/CD pipeline.
Docker	Containerization platform to build and run application containers.
Docker Hub	Cloud registry used to store and distribute Docker images (ramesh573/app).
GitHub	Source code management platform storing code, Dockerfile, and Jenkinsfile.
Nginx	Lightweight web server used inside the Docker container to serve the HTML page.

5. System Architecture

The architecture follows a linear CI/CD pipeline flow. Jenkins builds a Docker image from code, pushes it to Docker Hub, and then pulls and deploys the updated container on the same EC2 server — all automatically.

5.1 Architecture Flow

5.2 Flow Explanation

- Code on EC2 Server: Application code (index.html, Dockerfile, Jenkinsfile) is present on the EC2 server workspace used by Jenkins.
- Docker Build Image: Jenkins runs docker build --no-cache to create a fresh Docker image tagged as ramesh573/app.
- Push to Docker Hub: Jenkins logs in securely and pushes the image to the Docker Hub registry.
- Pull & Deploy Container: Jenkins stops the old running container, pulls the latest image, and starts a new container named 'app' on port 80.
- Web Output: The running Nginx container serves the HTML web page, accessible at the EC2 public IP on port 80.

6. Prerequisites

Before starting the implementation, ensure the following are in place:

6.1 Accounts and Software

- An AWS account with an EC2 instance (Ubuntu 22.04 LTS, t2.medium or higher).
- Security Group configured to allow inbound traffic on Port 22 (SSH), Port 8080 (Jenkins), and Port 80 (HTTP).
- Application code files (Dockerfile, Jenkinsfile, index.html) available on the EC2 server.
- A Docker Hub account with a repository to store the Docker image (ramesh573/app).
- Basic knowledge of Linux terminal commands and SSH access.

6.2 EC2 Instance Configuration

Parameter	Recommended Value
Instance Type	t2.medium (Jenkins + Docker)
OS / AMI	Ubuntu Server 22.04 LTS
Storage	20 GB (minimum)
Key Pair	Required for SSH access
Security Ports	22 (SSH), 8080 (Jenkins), 80 (Web)

7. Step-by-Step Implementation

Follow the steps below in order. All commands are to be run on the AWS EC2 Ubuntu terminal after SSH login.

Step 1: Connect to EC2 Instance

Open a terminal and connect to the EC2 instance using SSH:

```
ssh -i your-key.pem ubuntu@<your-ec2-public-ip>
```

Step 2: Update the System

```
sudo apt update -y
sudo apt upgrade -y
```

Step 3: Install Docker

Install Docker and add the Jenkins user to the Docker group:

```
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker jenkins
```

```
sudo usermod -aG docker ubuntu
docker --version
```

Step 4: Install Java (Required for Jenkins)

```
sudo apt install openjdk-17-jdk -y
java -version
```

Step 5: Install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install jenkins -y
```

Step 6: Start Jenkins

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl status jenkins
```

Step 7: Unlock Jenkins

Open a browser and navigate to: http://<your-ec2-ip>:8080

Retrieve the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Paste the password into the Jenkins setup wizard and install suggested plugins.

Step 8: Install Required Jenkins Plugins

Go to: Dashboard > Manage Jenkins > Plugins > Available Plugins. Search and install:

- Docker Pipeline
- Git Plugin
- Credentials Binding Plugin

Step 9: Add Docker Hub Credentials to Jenkins

Go to: Dashboard > Manage Jenkins > Credentials > System > Global Credentials > Add Credentials

Field	Value
Kind	Username with password
Username	Your Docker Hub username (ramesh573)
Password	Your Docker Hub password or access token
ID	dockerhub-creds
Description	Docker Hub Credentials

Step 10: Create a Jenkins Pipeline Job

Go to: Dashboard > New Item. Enter job name: docker-jenkins-pipeline, select Pipeline, and click OK.

1. Under Pipeline Definition, select: Pipeline script.
2. Paste the Jenkinsfile content directly into the script area.
3. Click Save and then Build Now to trigger the pipeline.

Step 11: Key Docker Commands Used in the Pipeline

Docker Build (no cache):

```
docker build --no-cache -t ramesh573/app .
```

Docker Login:

```
echo $PASS | docker login -u $USER --password-stdin
```

Docker Push:

```
docker push ramesh573/app
```

Deploy Container (stop old, run new):

```
docker stop app || true
docker rm app   || true
docker pull ramesh573/app
docker run -d -p 80:80 --name app ramesh573/app
```

8. Project Files Explanation

This project uses three key files. Each file is explained below with its full content and purpose.

8.1 Dockerfile

The Dockerfile defines the steps to build the Docker image. It uses the official Nginx Alpine image as the base and copies all project files into the Nginx web root directory.

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

Instruction	Meaning
FROM nginx:alpine	Use Nginx on Alpine Linux as the base image (small and efficient).
COPY . /usr/share/nginx/html	Copy all project files into Nginx's default web folder.

8.2 Jenkinsfile

The Jenkinsfile defines the complete CI/CD pipeline using Declarative Pipeline syntax. It contains 4 stages: Build Docker Image, Login to DockerHub, Push Image, and Deploy Container. The Deploy Container stage stops any previously running container, pulls the latest image, and starts a fresh container named 'app' on port 80. Docker Hub credentials are accessed securely using Jenkins credentials binding.

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "ramesh573/app"
    }

    stages {

        stage('Build Docker Image') {
            steps {
                sh 'docker build --no-cache -t $IMAGE_NAME .'
            }
        }

        stage('Login to DockerHub') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub-
creds',
                    usernameVariable: 'USER', passwordVariable: 'PASS')]) {
                    sh 'echo $PASS | docker login -u $USER --password-stdin'
                }
            }
        }
```

```
        stage('Push Image') {
            steps {
                sh 'docker push $IMAGE_NAME'
            }
        }

        stage('Deploy Container') {
            steps {
                sh '''
                docker stop app || true
                docker rm app || true
                docker pull $IMAGE_NAME
                docker run -d -p 80:80 --name app $IMAGE_NAME
                '''
            }
        }
    }
}
```

Stage	Description
Build Docker Image	Builds a fresh Docker image tagged ramesh573/app using --no-cache to avoid stale layers.
Login to DockerHub	Securely logs into Docker Hub using credentials stored in Jenkins.
Push Image	Pushes the built Docker image to Docker Hub for storage and access.
Deploy Container	Stops and removes the old container, pulls the latest image, and runs a new container named 'app' on port 80.

8.3 index.html

The index.html file is the web application served by Nginx inside the container. It displays a simple message confirming the DevOps pipeline is working:

```
<!DOCTYPE html>
<html>
  <head><title>DevOps Project</title></head>
  <body>
    <h1>DevOps Project Working &#128640;</h1>
    <h2>Version 1</h2>
  </body>
</html>
```

This is the exact content committed to the project, built into the Docker image, and rendered in the browser output shown in the screenshots.

9. Output Explanation

After the Jenkins pipeline executes successfully, several visible outputs confirm that the CI/CD process worked correctly. Real screenshots from the project are shown below.

9.1 GitHub Repository

The GitHub repository xramesh57/docker-jenkins-app contains the three project files: Dockerfile, Jenkinsfile, and index.html. The repository shows 27 commits, reflecting multiple iterative builds triggered by code changes.

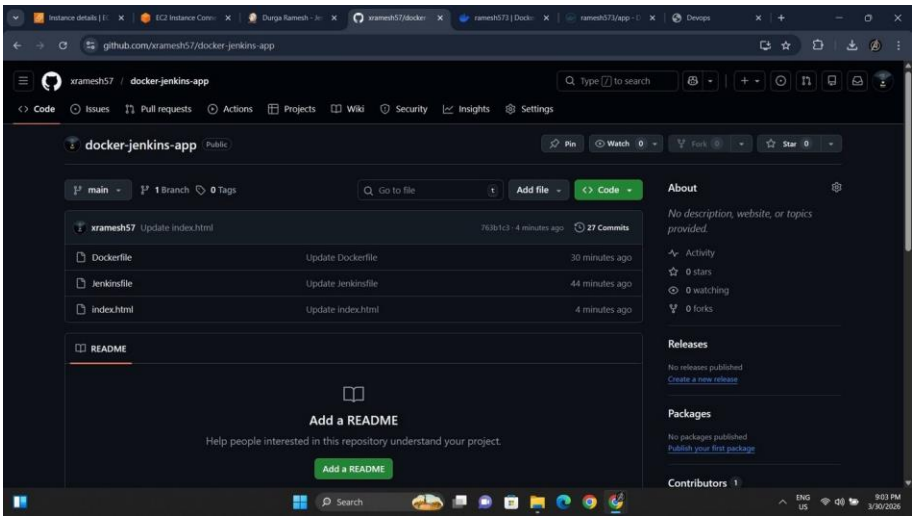

Figure 1: GitHub repository showing Dockerfile, Jenkinsfile, and index.html

9.2 Docker Hub — Image Tags (First Build)

After the first successful pipeline run, the Docker image ramesh573/app was pushed to Docker Hub with the 'latest' tag. The image has digest e031e03bfc14, runs on linux/amd64 architecture, and has a compressed size of 24.83 MB. The repository has 76 total pulls recorded.

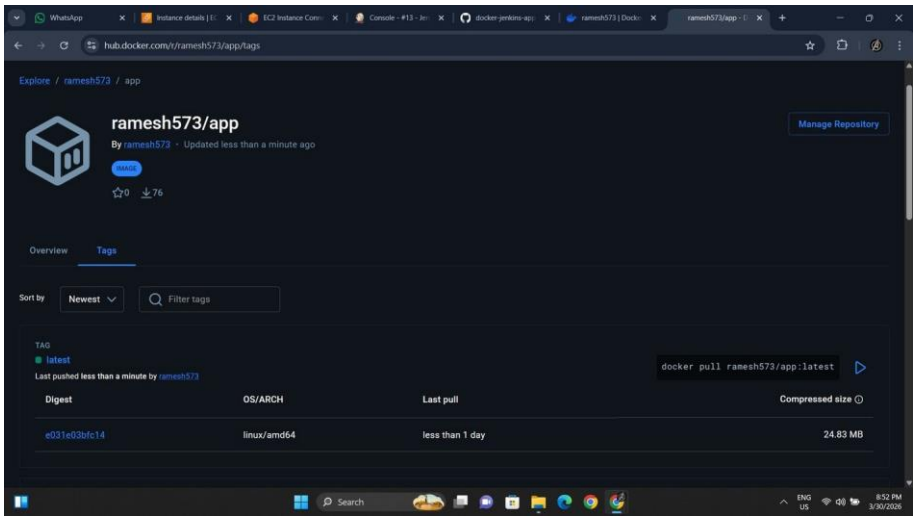

Figure 2: Docker Hub showing ramesh573/app:latest tag with digest e031e03bfc14 (24.83 MB)

9.3 Web Output — Version 1

After the Deploy Container stage ran successfully, the browser accessed http://15.134.219.182 and displayed the HTML web page. This confirms the container was deployed and is serving the Nginx page correctly.

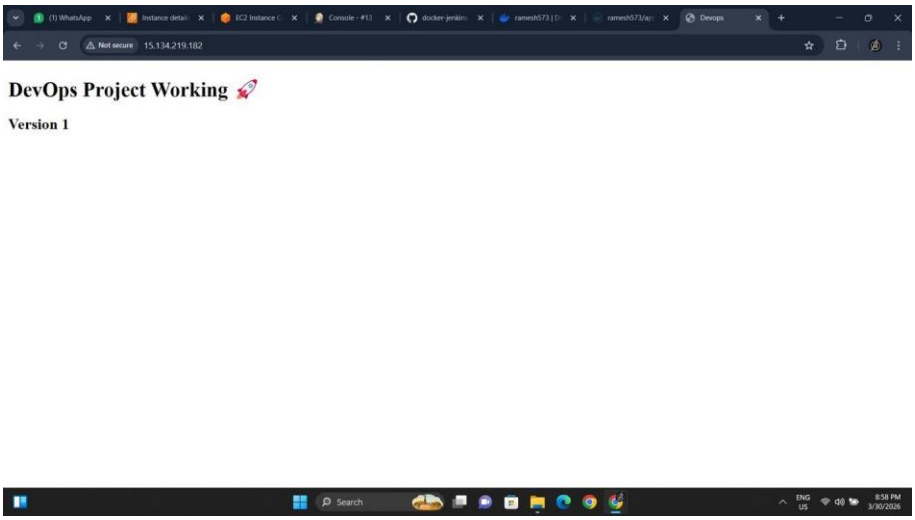

Figure 3: Browser showing 'DevOps Project Working' Version 1 at 15.134.219.182

9.4 Docker Hub — Multiple Tags After Re-runs

After additional pipeline runs triggered by code updates (index.html changed to Version 2), Docker Hub shows multiple tags. Tags 'latest' and '14' share digest 65a0bca5c2f7, while tag '13' has digest e031e03bfc14. This confirms the pipeline successfully built and pushed new images with each run.

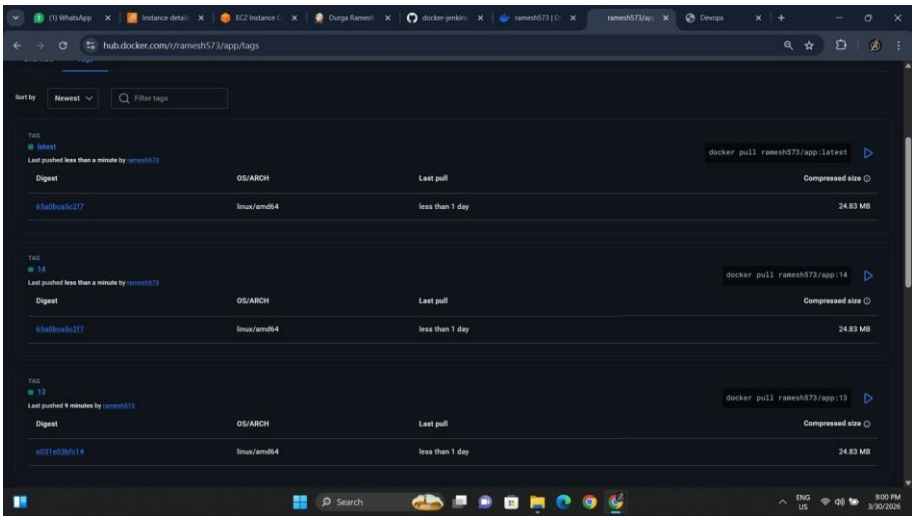

Figure 4: Docker Hub showing multiple tags — latest, 14, and 13 with their digests

9.5 Web Output — Version 2

After updating index.html to Version 2, the Jenkins pipeline was triggered. The Deploy Container stage stopped the old container, pulled the new image, and started a fresh container. The browser immediately showed Version 2, confirming complete end-to-end CI/CD automation.

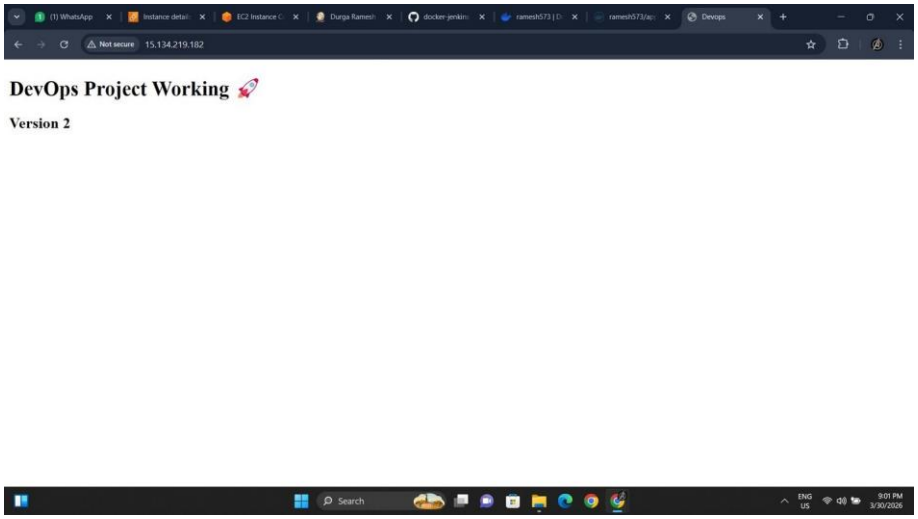

Figure 5: Browser showing 'DevOps Project Working' Version 2 — deployed by CI/CD pipeline

9.6 Result Summary

Test Case	Expected	Status
Jenkins pipeline executes all 4 stages	Success	PASS
Docker image built with --no-cache flag	Built	PASS
Image pushed to Docker Hub as ramesh573/app	Pushed	PASS
Old container stopped and new one deployed	Deployed	PASS
Web page accessible at http://15.134.219.182	Accessible	PASS
Version 2 update triggers new build and redeploy	Updated	PASS

10. Advantages of the Project

- Full Automation: The entire process from code change to live deployment is automated. No manual steps are required after initial setup.
- End-to-End CI/CD: The pipeline covers build, push, and automated deployment — making it a complete CI/CD workflow.
- Consistency: Docker ensures the application behaves identically in development, testing, and production environments.
- Portability: The Docker image can be pulled and run on any machine or cloud platform supporting Docker.
- Security: Docker Hub credentials are stored securely in Jenkins. Passwords are never exposed in the Jenkinsfile.
- Reusability: The same pipeline structure can be reused for different projects with minor modifications.
- Version Control: Both application code and pipeline script are version-controlled, supporting traceability and rollback.
- Scalability: The pipeline can be extended to support multiple environments, testing stages, and deployment strategies.

11. Limitations

- No Automated Testing: The pipeline does not include unit or integration tests. A broken application could still be built and deployed.
- Brief Downtime: Stopping the container before starting a new one causes a brief period of unavailability during redeploy.
- Single Tag: The pipeline always pushes using the 'latest' tag, making rollback to a specific previous version difficult.
- No Notifications: The pipeline does not send alerts via email or Slack when a build or deployment fails.
- No Load Balancing: A single container is deployed on one server. There is no auto-scaling or load balancing.
- HTTP Only: The application runs over HTTP without SSL/TLS, making it unsuitable for production without additional configuration.

12. Conclusion

This project successfully demonstrated the implementation of a complete CI/CD pipeline using Jenkins and Docker on AWS EC2. The pipeline automates the full workflow: building a Docker image with the --no-cache flag, pushing it to Docker Hub, and automatically deploying an updated container — all without manual intervention.

The inclusion of the Deploy Container stage upgrades this from a basic CI pipeline to a true CI/CD pipeline. Real project outputs — including Docker Hub image tags with multiple versioned builds and browser screenshots of the live web application — confirm that the pipeline works end-to-end.

This project provides hands-on experience with Jenkins, Docker, and AWS EC2 — all widely used tools in modern software development and DevOps teams.

13. Future Enhancements

13.1 Rolling Updates and Zero-Downtime Deployment

The current deploy stage causes brief downtime. A blue-green or rolling deployment strategy can ensure zero downtime by running two versions simultaneously and switching traffic only when the new one is healthy.

13.2 Kubernetes Orchestration

The Docker image pushed to Docker Hub can be deployed to a Kubernetes cluster (e.g., Amazon EKS or minikube). Kubernetes provides auto-scaling, self-healing, rolling updates, and load balancing — making this a production-grade solution.

13.3 Automated Testing Stage

A testing stage can be added before the build stage to run unit tests. The pipeline will only proceed to build and deploy if all tests pass, ensuring code quality at every step.

13.4 Versioned Image Tags

Instead of always using the 'latest' tag, images can be tagged with the Jenkins build number (e.g., ramesh573/app:14). This supports version history and easy rollback to any specific build.

```
sh 'docker build --no-cache -t $IMAGE_NAME:$BUILD_NUMBER .'
```

13.5 HTTPS with Nginx Reverse Proxy

An Nginx reverse proxy can be configured in front of the container with SSL/TLS certificates to enable HTTPS, making the application production-ready and secure.

13.6 Build Notifications

Jenkins can be configured to send build and deployment status notifications via email or Slack, enabling teams to respond to failures immediately.

References

1. Jenkins Official Documentation. Available: https://www.jenkins.io/doc/
2. Docker Official Documentation. Available: https://docs.docker.com/
3. Docker Hub — ramesh573/app. Available: https://hub.docker.com/r/ramesh573/app
4. GitHub Repository — xramesh57/docker-jenkins-app. Available: https://github.com/xramesh57/docker-jenkins-app
5. AWS EC2 Documentation. Available: https://docs.aws.amazon.com/ec2/
6. Nginx Official Documentation. Available: https://nginx.org/en/docs/